# Informatique Théorique

## CM6: Représentation des réels à virgule flottante

Florian Bridoux

Polytech Nice Sophia

2025-2026

# Table des matières

1 Virgule flottante

2 Arithmétique en virgule flottante

# Table des matières

1 Virgule flottante

2 Arithmétique en virgule flottante

# Rappel : Problèmes de la représentation en virgule fixe

- La représentation en virgule fixe impose une plage limitée de valeurs absolues (grandes ou petites).
- Exemples : - Si l'on réserve $m = 32$ bits pour la partie entière et $n = 32$ bits pour la partie fractionnaire, alors :

$$\text{Valeur maximale} = 2^{32} - 2^{-32}.$$

$$\text{Valeur minimale} (> 0) = 2^{-32}$$

- Pour représenter la plage complète des `float` simple précision en Java ($10^{38}$ à $10^{-45}$), il faudrait environ 380 bits $(m + n)$ contre 32.

Solution ? Les virgules flottantes.

# Représentation en virgule flottante

- Un nombre réel est représenté par :
  - un **signe** $s$ : sur 1 bit (0 pour positif, 1 pour négatif),
  - une **mantisse** (en base $b$) $m$,
  - un **exposant** $e$.
- La valeur du réel est donnée par :

$$(-1)^s \cdot m \cdot b^e$$

- **Exemple** : représentation de $\pi$ :
  - $(-1)^0 \cdot 0{,}031 \cdot 10^2$
  - $(-1)^0 \cdot 3{,}142 \cdot 10^0$
  - $(-1)^0 \cdot 31{,}42 \cdot 10^{-1}$
  - $(-1)^0 \cdot 0{,}003 \cdot 10^3$
- **Observations** :
  - Plusieurs représentations approchées sont possibles.
  - Elles n'ont pas toutes la même précision.

# Représentation normalisée (et notation scientifique)

- Dans une base $b$, une représentation est **normalisée** si elle est sous la forme :

$$(-1)^s \cdot M \cdot b^e$$

avec $M \in [1, b[$.

- **En base 2** :
  - $M = 1,M'$ (le premier chiffre de $M$ est toujours 1).
  - On stocke $M'$, appelée **pseudo-mantisse**.
- **Observations** :
  - Tout réel (non nul) a une **unique notation scientifique**.
  - Mais tout réel (non nul) n'a pas nécessairement une **représentation normalisée exacte** :
    - si $N_m$ bits représentent la mantisse,
    - et $N_e$ bits représentent l'exposant $e$.
  - Étant donnés $N_m$ et $N_e$, la représentation normalisée offre la **meilleure précision possible** pour ce réel.
- **Cas particulier** : 0 n'a pas de représentation normalisée !

# Exemples de représentation normalisée

**Base 10 :**

- Nombre : $314,2$
- Normalisé : $3,142 \cdot 10^2$ $(M = 3,142,\ e = 2,\ b = 10)$

**Base 2 :**

- Nombre : $(13,5)_{10} = (1101,1)_2$
- Normalisé : $(1,1011)_2 \cdot 2^3$ $(M = (1,1011)_2,\ e = 3,\ b = 2)$

**Remarque :** Le nombre 0 n'a pas de représentation normalisée.

# Le système IEEE 754

**Standard de représentation des nombres en virgule flottante binaire :**

- Forme générale : `S | Exposant | Pseudo-mantisse`
- Taille des champs :
  - $S$ : 1 bit (signe)
  - Exposant : $N_e$ bits
  - Pseudo-mantisse : $N_m$ bits

**Exemples :**

- **Simple précision** : $1 + 8 + 23 = 32$ bits. Utilisé pour `float` en Java.
- **Double précision** : $1 + 11 + 52 = 64$ bits. Utilisé pour `double` en Java.

# Codage de l'exposant

**Problème :** l'exposant peut être négatif.
**Solution :**

- Utilisation d'une **translation par un biais** au lieu du complément à deux.
- Plage initiale : $[0, 2^{N_e} - 1]$.
- Après translation : $\left[ -2^{N_e-1} + 1, 2^{N_e-1} \right]$ avec un biais $2^{N_e-1} - 1$.

**Exemples :**

- Biais de 127 pour la simple précision ($N_e = 8$).
- Biais de 1023 pour la double précision ($N_e = 11$).

# Exemple avec 8 bits

**Configuration des bits :**

- 1 bit pour le signe ($S$)
- 3 bits pour l'exposant ($E$)
- 4 bits pour la pseudo-mantisse ($M$)

**Valeur binaire :** 0 1 1 0 1 0 1 1

- $S = 0$ : nombre positif.
- Exposant $E = 3$ : 6 (valeur stockée) $-$ **biais** 3.
- Pseudo-mantisse $M = 1011$.
- Mantisse réelle : 1,1011.

**Représentation en base 2 :**

$$1{,}1011 \times 2^3 = 1101{,}1$$

**Représentation en base 10 :**

$$1101{,}1 = 13{,}5$$

# Représentation du réel 3,125

**Représentation en base 2 :**

$$3,125_{10} = 11,001_2$$

**Normalisation :**

$$11,001_2 = 1,1001 \times 2^1$$

**Représentation normalisée :**

$$+1,1001 \times 2^1$$

**Codage IEEE 754 (8 bits) :**

- **Signe :** 0 (positif)
- **Exposant :** 1 (excentrement de 3 : $1 + 3 = 4$, codé sur 3 bits)
- **Pseudo-mantisse :** 1001 (bits significatifs après la virgule, le bit caché est implicite)

**Bits représentés :**

0 1 0 0 1 0 0 1

# Représentation du zéro en virgule flottante

**Problème de la normalisation :**

- Il n'est pas possible de représenter le zéro avec une écriture normalisée.
- On réserve donc le plus petit et le plus grand des exposants pour des écritures non normalisées (dont 0).

# Codage de l'exposant, revisité pour inclure des nombres non normalisés

**Codage de l'exposant :**

- On passe de $[0, 2^{N_e} - 1]$ à $[-2^{N_e-1} + 1, 2^{N_e-1}]$ en translatant de $2^{N_e-1} - 1$.
- **Réservations des exposants :**
  - Le plus grand exposant, codé 1..1, représente les valeurs spéciales :
    - **L'infini** : Pseudo-mantisse à $000\ldots0$ ; le signe détermine si l'infini est positif ou négatif.
    - **NaN (Not A Number)** : Pseudo-mantisse différente de $000\ldots0$.
  - Le plus petit exposant, codé 0..0, représente :
    - **Zéro** : Pseudo-mantisse à $000\ldots0$ ; il existe deux codages (positif ou négatif).
    - Autres nombres dénormalisés avec mantisse non nulle...
- Les nombres normalisés ont donc un intervalle d'exposant possible un peu plus réduit...

# Table des matières

1 Virgule flottante

2 Arithmétique en virgule flottante

# Addition de deux flottants positifs

1. Décaler à droite la mantisse (sans oublier le bit caché) du nombre possédant le plus petit exposant jusqu'à arriver à l'exposant de l'autre nombre.
2. Additionner les mantisses.
3. Normaliser le résultat.
4. Arrondir.

# Exemple d'addition de deux flottants

- Addition des deux flottants :

$$0\ 100\ 10 + 0\ 011\ 10 \quad (3 + 1{,}5)$$

- Représentation des nombres :

$$1,1 \times 2^1 + 1,1 \times 2^0$$

- Étapes de l'addition :
  1. $1,10 \times 2 + 0,11 \times 2$
  2. $10,01 \times 2$
  3. $1,001 \times 2^2$
  4. $1,00 \times 2^2$ (c'est-à-dire 4 !) car on n'a que deux chiffres après la virgule.
- Résultat : 0 101 00
- Il n'y aurait pas eu d'erreur d'arrondi si $N_m \geq 3$.

# Addition de 5 et 8 en virgule flottante

Additionner $5 + 8$ en représentation en virgule flottante :

- Représentation en base 2 de 5 : $1,01 \times 2^2$
- Représentation en base 2 de 8 : $1,00 \times 2^3$

**Alignement des exposants :**

- Décaler la mantisse de 5 : $1,01 \times 2^2 \rightarrow 0,101 \times 2^3$

**Addition des mantisses :**

$$0,101 + 1,00 = 1,101$$

**Normalisation :**

$1,101$ est déjà normalisé.

**Arrondi (mantisse sur 2 bits) :**

$$1,101 \approx 1,10$$

**Résultat final :**

$$1,10 \times 2^3 = 12$$

# Addition de 8 et 8 en virgule flottante

Additionner $8 + 8$ en représentation en virgule flottante :

- Représentation en base 2 de 8 : $1,00 \times 2^3$
- Représentation en base 2 de 8 : $1,00 \times 2^3$

**Alignement des exposants :**

- Les exposants sont déjà égaux, donc pas besoin de décalage.

**Addition des mantisses :**

$$1,00 + 1,00 = 10,00$$

**Normalisation :**

$$10,00 \rightarrow 1,0 \times 2^4$$

**Overflow :**

- Le résultat dépasse la plus grande valeur pouvant être représentée avec la précision donnée (overflow).

**Résultat final :**

# Arrondi en virgule flottante

- L'arrondi se fait à la valeur la plus proche possible.
- En cas d'équidistance (c'est-à-dire lorsque deux valeurs sont à égale distance de la valeur réelle), on arrondit vers la valeur paire la plus proche.

**Exemple :**

- 4.5 arrondi à un chiffre après la virgule donne 4.0 (car 4.0 est pair).
- 5.5 arrondi à un chiffre après la virgule donne 6.0 (car 6.0 est pair).

# Multiplication de flottants

- Déterminer le signe du résultat.
- Calculer la somme des exposants (n'oubliez pas de corriger l'excentrement).
- Multiplier les mantisses (en virgule fixe !).
- Gérer les arrondis.
- Vérifier les dépassements de capacité : **overflow** ou **underflow**.

**Remarque :** Multiplier par la base $b$ revient à ajouter un à l'exposant.

# Avantages de la norme IEEE 754

- Les nombres flottants sont portables d'un système à l'autre.
- Les opérations sont uniquement définies.
- Les programmes ont un comportement unique.
- Les programmes numériques sont analysables.